

# 지식그래프를 AstraDB에 NoSQL로 담는 법

그래프 DB가 아니라 문서형 NoSQL(Data API) 컬렉션 하나에 엔티티와 관계를 어떻게 저장하고, 어떻게 다시 그래프로 되살려 쓰는지 — 라이브 데이터로 설명합니다.

AstraDB · DataStax

JSON Data API

단일 컬렉션 + kind 필드

앱-사이드 1-홉 탐색

2

컬렉션  
kg · doc\_registry

49

엔티티(노드)  
kind = "entity"

51

엣지(관계)  
kind = "edge"

8

적재 문서  
한·영 코퍼스

```
# Astra keyspace: default_keyspace
default_keyspace
├─ kg // 지식그래프 = 엔티티 + 엣지 (한 컬렉션에 같이)
│   ├── {kind:"entity"} × 49 노드
│   └─ {kind:"edge"} × 51 관계(트리플)
└─ doc_registry // 적재 문서 추적 × 8
```

핵심: 테이블도 그래프 엔진도 없습니다. 노드와 엣지를 같은 컬렉션에 넣고 kind 필드 하나로 구분합니다. 그래프 순회(Cypher/Gremlin)는 쓰지 않고, 질의 때 앱이 메모리에서 1-홉을 구성합니다.

## 한 컬렉션, 두 종류의 문서

모든 KG 데이터는 **평범한 JSON 문서**입니다. **kind** 값이 **entity**면 노드, **edge**면 관계(트리플)입니다. 둘 다 **doc\_id**로 "어느 원문에서 나왔는지"를 추적하고, **norm** 계열 필드로 표기 흔들림을 흡수합니다.

**kind: "entity"** 엔티티 — 노드

개념·기관·기술 등 "무엇"

**kind: "edge"** 엣지 — 관계 트리플

(출발)—관계→(도착) "어떻게 연결"

### A 엔티티 문서 (실데이터)

```
{
  "kind": "entity",
  "name": "이상거래탐지시스템 (FDS)", // 원문 표기 그대로 (표시용)
  "norm": "이상거래탐지시스템",      // 정규화 키 - 매칭·중복병합용
  "type": "technology",              // LLM이 분류한 종류
  "doc_id": "98b3454c..."          // 출처 문서 (registry와 연결)
}
```

### B 엣지 문서 — 트리플 (실데이터)

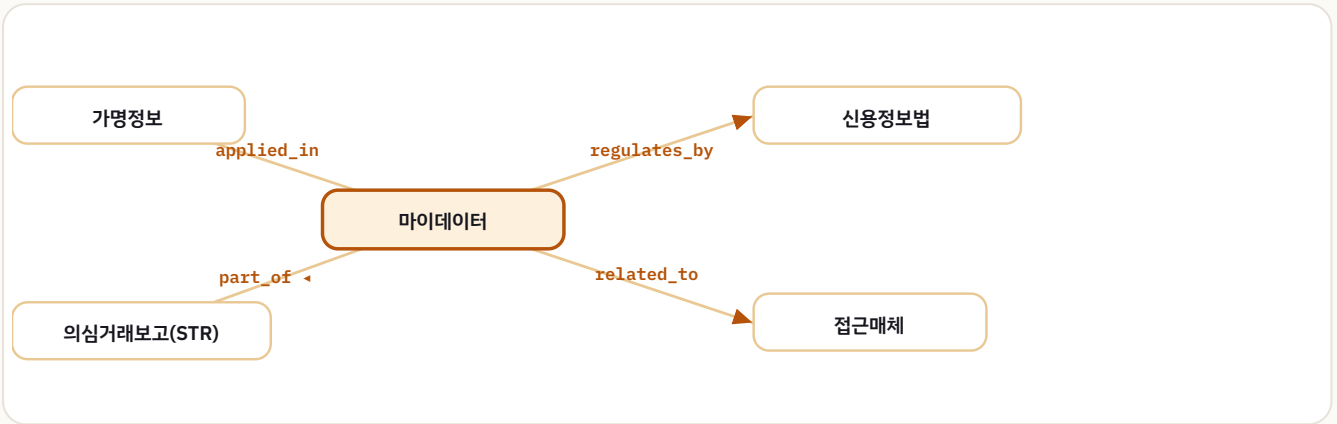
```
{
  "kind": "edge",
  "src": "마이데이터", "rel": "regulates_by", "dst": "신용정보법",
  // └─ 출발 노드 ──── 관계 라벨 ──── 도착 노드
  "src_norm": "마이데이터", "dst_norm": "신용정보법", // 정규화 트리플
  "doc_id": "51582e40..."
}
```

→ 읽으면 "마이데이터는 신용정보법의 규율을 받는다". 이런 행(=문서)이 51개 나열된 **엣지 리스트**가 곧 그래프입니다.

왜 **norm**이 따로 있나? "마이데이터"와 "마이데이터 사업자", "FDS"와 "이상거래탐지시스템(FDS)"를 한 **노드**로 묶기 위해서입니다. 매칭·중복제거는 항상 정규화 키로 하고, 화면에는 원문 표기(**name**)를 보여줍니다.

## 실제로 들어있는 그래프

아래는 운영 중인 AstraDB에서 그대로 가져온 트리플입니다. 평평한 엣지 리스트지만, `src_norm`으로 모으면 이렇게 연결됩니다.



### 엣지 리스트 (원본)

마이데이터 **regulates\_by**→ 신용정보법  
 마이데이터 **related\_to**→ 접근매체  
 가명정보 **applied\_in**→ 마이데이터·데이터결합  
 의심거래보고(STR) **part\_of**→ 특정금융정보법  
 Docling **requires**→ Ruff  
 Langflow **can\_be\_deployed\_using**→ Docker

### 관계 라벨 분포

|                 |                        |     |
|-----------------|------------------------|-----|
| applies_to      | <div><div></div></div> | 7   |
| related_to      | <div><div></div></div> | 5   |
| integrates_with | <div><div></div></div> | 5   |
| regulates       | <div><div></div></div> | 4   |
| part_of         | <div><div></div></div> | 3   |
| requires        | <div><div></div></div> | 2   |
| + 22 종          | <div><div></div></div> | 각 1 |

관계 라벨(**rel**)은 LLM이 자유 추출하되 **snake\_case** 통제 어휘로 정규화합니다(한글 라벨 → 영문 표준화). 그래서 같은 의미의 관계가 제각각 흩어지지 않고 위처럼 집계됩니다.

# 왜 이렇게 했고, 어떻게 동작하나

## # 설계 선택 3가지

### 1 그래프 DB를 쓰지 않는다

AstraDB Data API(문서형 NoSQL)에 JSON으로 저장. Gremlin·Cypher 같은 그래프 순회 엔진이 없습니다. 운영 부담이 적고, RAG에 필요한 건 작은 1-홉 이웃이라 충분합니다.

### 2 탐색은 앱이 메모리에서 한다

질의 때 `kg` 컬렉션을 전부 로드 → 파이썬에서 `kind` 로 노드/엣지를 나누고, 질문 관련도로 점수를 매겨 1-홉 서브그래프를 만듭니다. KG가 100건으로 작기에 가능한 단순함입니다.

### 3 정규화 필드로 노드를 병합한다

`norm` · `src_norm` · `dst_norm` 이 표기 흔들림을 흡수해 "마이데이터 = 마이데이터 사업자"를 한 노드로, 중복 엣지를 한 줄로 정리합니다.

## → graph 질의가 답이 되기까지

1 라우터 — granite LLM이 질문을 보고 `{"graph": true}` 로 그래프 도구를 선택

2 전체 로드 — `kg` 컬렉션을 페이지네이션으로 모두 읽어 노드/엣지로 분리

3 관련도 정렬 — 질문 토큰·벡터 시드와 겹치는 엔티티/엣지에 가점, 점수>0만 유지

4 1-홉 구성 — 정규화 키로 중복·자기참조 제거 → 엔티티 칩 + 엣지 트리플로 정리

5 합성 — 이 서브그래프를 컨텍스트로 granite가 답변 생성, UI는 같은 서브그래프를 근거 패널로 노출

요약 — AstraDB에는 "노드와 엣지를 적은 JSON 문서"가 한 컬렉션에 평평하게 들어있을 뿐입니다. 그래프의 "연결"은 저장이 아니라 질의 시점에 앱이 정규화 키로 이어 붙여 만들어집니다. 작은 도메인 KG에 맞는, 단순하고 설명 가능한 설계입니다.